SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Madhusarvesh *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Madhusarvesh

Analytical Problem Solving

Assessment 1 - Complete Solutions (CSA1710 Artificial Intelligence)

Question 1: Water Jug Problem (4-gallon and 3-gallon jugs)

1. Problem understanding

State: (x, y) where x = water in the 4-gallon jug (0 to 4) and y = water in the 3-gallon jug (0 to 3).
Initial state: (0, 0). **Goal state:** (2, y) for any y (here the 4-gallon jug holds exactly 2).
Assumptions: a jug can be filled from the pump, emptied onto the ground, or poured into the other jug; there is no measuring device, so a pour ends only when the source is empty or the target is full.

2. Method selection

The problem is modelled as a state-space search. Breadth-First Search (BFS) is chosen because every action has equal cost and BFS is complete and returns the shortest sequence of actions.

Production rules (operators)

#	Rule	Condition
1	(x, y) -> (4, y) Fill the 4-gal jug	x < 4
2	(x, y) -> (x, 3) Fill the 3-gal jug	y < 3
3	(x, y) -> (0, y) Empty the 4-gal jug	x > 0
4	(x, y) -> (x, 0) Empty the 3-gal jug	y > 0
5	(x, y) -> (4, y - (4 - x)) Pour 3-gal into 4-gal until the 4-gal is full	x + y >= 4, y > 0
6	(x, y) -> (x - (3 - y), 3) Pour 4-gal into 3-gal until the 3-gal is full	x + y >= 3, x > 0
7	(x, y) -> (x + y, 0) Pour all of the 3-gal into the 4-gal	x + y <= 4, y > 0
8	(x, y) -> (0, x + y) Pour all of the 4-gal into the 3-gal	x + y <= 3, x > 0

3. Solution process (one shortest solution, 6 actions)

Step	Action	State (4-gal, 3-gal)
0	Start	(0, 0)
1	Fill the 3-gallon jug from the pump	(0, 3)
2	Pour the 3-gallon jug into the 4-gallon jug	(3, 0)
3	Fill the 3-gallon jug again	(3, 3)
4	Pour from the 3-gal jug into the 4-gal jug until it is full (only 1 gallon fits)	(4, 2)
5	Empty the 4-gallon jug onto the ground	(0, 2)
6	Pour the 3-gallon jug (2 gallons) into the 4-gallon jug	**(2, 0) GOAL**

Alternative solution (also 6 actions): (0,0) -> (4,0) -> (1,3) -> (1,0) -> (0,1) -> (4,1) -> (2,3). Here the 4-gallon jug also ends with exactly 2 gallons.

4. Accuracy and interpretation

- Check for solution 1: after step 3 the total water is 6 gallons; step 4 moves 1 gallon so (4,2) holds 4 + 2 = 6. Throwing away the 4 gallons leaves 2, which is moved into the 4-gallon jug.
- The key idea is that 3 - (4 - 3) = 2: filling the 3-gallon jug twice and topping up the 4-gallon jug leaves a remainder of exactly 2 gallons.
- Because BFS explores states level by level, no solution with fewer than 6 actions exists.

Question 2: Mars Rover - PEAS Analysis

Problem understanding

The Mars Rover is an autonomous robotic agent that moves across the Martian surface, selects rocks and soil, analyses them with onboard instruments and sends the results to Earth. The agent is described with the **PEAS** framework (Performance measure, Environment, Actuators, Sensors).

(i) Percepts for this agent

- **Vision:** stereo/panoramic camera images of terrain, rocks and soil; hazard-avoidance camera images.
- **Scientific instruments:** spectrometer readings (chemical and mineral composition), X-ray/laser readings, moisture and soil-property data.
- **Navigation and self-state:** wheel encoders, gyroscope/inertial unit (tilt and slope), position/heading estimate, wheel-slip and motor-current signals.
- **Environmental:** temperature, wind, dust and radiation levels, sunlight/solar-panel output.
- **Internal status:** battery level, component temperature and health, memory and data storage.
- **Communication:** commands and updated targets received from Earth (via orbiters).

(ii) Characterization of the operating environment

Property	Value	Reason
Observability	Partially observable	The camera sees only a limited area; the sub-surface and far terrain are unknown.
Agents	Single agent	Only the rover acts (human operators on Earth only give high-level goals).
Determinism	Stochastic	Wheel slip, sand, and uncertain sensor readings mean actions do not always give the expected result.
Episodic / Sequential	Sequential	Current moves affect later options (energy used, position, sample collected).
Static / Dynamic	Dynamic	Dust storms, changing light and temperature occur while the rover is deciding.
Discrete / Continuous	Continuous	Position, speed, temperature and sensor values vary continuously.
Known / Unknown	Largely unknown	Terrain is not fully mapped; the rover must learn it while exploring.

(iii) Actions the agent can take (actuators)

- Drive forward/backward, turn, climb, stop, and select a path around obstacles.
- Move the robotic arm; scoop, drill or scrape soil and rock; place a sample in the analysis instrument.
- Rotate/tilt the mast camera; capture images; switch instruments on or off.
- Transmit data to Earth/orbiter; receive and store commands.
- Manage power: recharge, enter sleep or safe mode, adjust solar panels.

(iv) Performance measure

- **Science value:** number and quality of samples analysed; discovery of useful evidence (water, organic compounds).
- **Data return:** amount of valid data successfully transmitted to Earth.
- **Coverage and efficiency:** area explored per unit of energy and time.
- **Safety and survival:** no collisions, falls or stuck wheels; mission lifetime and battery health maintained.

A combined score can be used, for example: Performance = w1 x (valuable samples) + w2 x (data transmitted) + w3 x (area explored) - w4 x (energy used) - w5 x (risk events).

(v) Most suitable agent architecture and why

A **utility-based, learning agent** built as a **hierarchical (layered) architecture** is most suitable.

- **Reflex layer (low level):** immediate responses such as stopping when a cliff or steep tilt is sensed. This is needed because radio delay from Earth is several minutes.
- **Goal/planning layer:** plans routes to target rocks using search (A*) and a map built from the percepts.
- **Utility layer:** trades off conflicting goals - science value vs. energy, time and risk - and picks the best action among many good ones.
- **Learning element:** improves terrain classification and sample selection with experience, since the environment is partially observable, stochastic and unknown.
- A simple reflex or model-based reflex agent alone is not enough because the task needs foresight, memory of the map, and balancing multiple objectives.

Question 3: 8-Queens Problem - Search Space and Problem Formulation

1. Problem understanding

Two queens attack each other if they share a row, a column or a diagonal. A valid solution therefore has exactly one queen in each row and each column and no two queens on a common diagonal. In the given figure, the first-column and last-column queens lie on one diagonal (they are 7 columns apart and 7 rows apart), so the configuration fails the goal test even though it looks complete.

2. Problem formulation (components)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board. (Efficient version: one queen per column, queens placed left to right, each in a non-attacked square.)
Initial state	Empty board with no queens.
Actions	Add a queen to the leftmost empty column in any row that is not attacked by a queen already placed. (Complete-state version: move a queen within its column.)
Transition model	Result(s, a) returns the board with the new queen added in the chosen square.
Goal test	8 queens are on the board and no two attack each other (distinct rows, distinct columns, distinct diagonals).
Path cost	Not important here - only the final arrangement matters (each placement may be given cost 1).

3. Exploring the search space

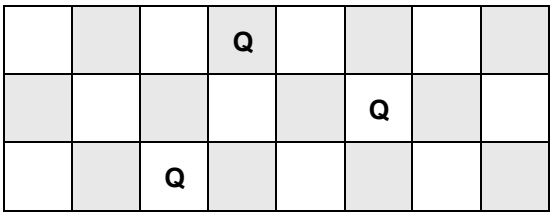
Formulation	Size of search space
Any 8 queens on any of 64 squares (combinations)	C(64,8) = about 4.4 x 10^9
Add queens one at a time on any square (sequences)	64 x 63 x ... x 57 = about 1.8 x 10^14
One queen per column, any row	8^8 = 16,777,216
One queen per row and per column (permutations)	8! = 40,320
Incremental, only non-attacked squares allowed	Only 2,057 states

Strategies: (a) **Depth-first search with backtracking** - place a queen in column 1, then column 2 in a safe row, and so on; when a column has no safe row, backtrack. This is the standard and most efficient systematic method. (b) **BFS** works but stores many partial boards, so memory is large. (c) **Local search** (hill climbing, min-conflicts, simulated annealing) starts with all 8 queens on the board and repeatedly moves one queen to reduce the number of attacking pairs; it solves even very large n quickly.

4. Solution and interpretation

The puzzle has **92 solutions** (12 distinct ones after removing rotations and reflections). One valid solution lists the row of the queen in columns 1 to 8 as **(1, 5, 8, 6, 3, 7, 2, 4)**:

Q							
						Q	
				Q			
							Q
	Q						



Verification: rows 1,5,8,6,3,7,2,4 are all different. Row-minus-column values (0,3,5,2,-2,1,-5,-4) are all different and row-plus-column values (2,7,11,10,8,13,9,12) are all different, so no two queens share a diagonal. The first-column queen (row 1) and last-column queen (row 4) are not on the same diagonal, unlike the figure.

Question 4: OLA Cab Booking - Type of Agent and Pseudocode

1. Problem understanding

PEAS	Description
Performance	Customer satisfaction, trip comfort, low fare, short waiting time/ETA, safety, successful booking.
Environment	OLA app, map and traffic data, available drivers and cab types, fares and surge pricing, customer preferences. Partially observable, stochastic (traffic, driver cancellations), dynamic, sequential.
Actuators	Display options, select/confirm cab type, assign driver, show route, send notifications, process payment.
Sensors	Pickup and drop location (GPS), user selection and comfort preference, traffic data, driver location, fare estimate.

2. Type of agent

A **utility-based agent** (a goal-based agent with a utility function) is most suitable. The goal is to reach the destination, but several cab types can reach it; they differ in comfort, price and waiting time. A pure goal-based agent only knows "reached / not reached", so it cannot choose the best option. The utility function scores each cab type using the customer's comfort preference, fare, and ETA, and the agent picks the highest-utility option.

Utility of a cab type c: U(c) = w1 x Comfort(c) - w2 x Fare(c) - w3 x ETA(c) + w4 x Rating(c), where w1..w4 are weights set from the customer's preferences.

3. Pseudocode

```
function OLA-UTILITY-AGENT(percept) returns action
  persistent: state, preferences, availableCabs

  state <- UPDATE-STATE(state, percept)    // pickup, drop, traffic, drivers
  source, destination <- state.pickup, state.drop

  if source or destination is invalid then
      return REQUEST-VALID-LOCATION()

  availableCabs <- GET-NEARBY-CABS(source)
  if availableCabs is empty then
      return NOTIFY("No cabs available, try later")

  distance, time <- ESTIMATE-ROUTE(source, destination)

  bestCab <- NULL
  bestUtility <- -infinity
  for each cabType in {mini, micro, sedan, shared, prime}:
      if cabType is available:
          fare <- CALCULATE-FARE(cabType, distance, time, surge)
          eta  <- ESTIMATE-ARRIVAL(cabType, source)
          comfort <- COMFORT-SCORE(cabType)
          U <- w1*comfort - w2*fare - w3*eta + w4*RATING(cabType)
          if U > bestUtility then
              bestUtility <- U
              bestCab <- cabType

  DISPLAY(all cab options with fare and ETA, highlight bestCab)
  choice <- GET-CUSTOMER-CHOICE(default = bestCab)
  driver <- ASSIGN-DRIVER(choice, source)

  if driver accepts then
      return CONFIRM-BOOKING(choice, driver, fare)
```

```
else
    remove choice from availableCabs; repeat from selection step
```

4. Interpretation

The agent senses the trip request, generates the available cab types, scores each by utility, shows the ranked options and lets the customer confirm. If the driver declines, the agent re-plans with the next-best option, which shows its ability to handle a dynamic, uncertain environment.

Question 5: Uniform Cost Search (UCS) on the Delivery Network

1. Problem understanding

Edges read from the graph (directed, with cost = fuel + time):

Edge	S-A	S-G	A-B	A-C	B-D	C-D	C-G	D-G
Cost	1	12	3	1	3	1	2	3

2. Method selection

UCS always expands the frontier node with the lowest path cost g(n) using a priority queue. Since all costs are positive, UCS is **complete and optimal**, which is exactly what a least-cost routing problem requires. The goal test is applied when a node is **removed** from the queue (not when generated).

3. Solution process

Step	Node expanded (g)	Nodes generated / updated	Frontier (priority queue, ordered by g)
1	S (0)	A = 1, G = 12	A(1), G(12)
2	A (1)	B = 1+3 = 4, C = 1+1 = 2	C(2), B(4), G(12)
3	C (2)	D = 2+1 = 3, G = 2+2 = 4 (better than 12, so updated)	D(3), B(4), G(4)
4	D (3)	G via D = 3+3 = 6 (worse than 4, ignored)	B(4), G(4)
5	B (4)	D via B = 4+3 = 7 (worse than 3, ignored)	G(4)
6	G (4)	Goal test passed - stop	-

Order of expansion: S, A, C, D, B, G. (B and G tie at cost 4; B is removed first because it was inserted earlier. Choosing G first gives the same answer.)

4. Accuracy and interpretation

Optimal path: S -> A -> C -> G with minimum total cost = 1 + 1 + 2 = 4.

Path	Cost
S -> A -> C -> G	1 + 1 + 2 = **4 (optimal)**
S -> A -> C -> D -> G	1 + 1 + 1 + 3 = 6
S -> A -> B -> D -> G	1 + 3 + 3 + 3 = 10
S -> G (direct)	12

The direct route S -> G looks shortest in number of edges (1 edge) but is the most expensive. UCS correctly prefers the cheaper multi-hop route through A and C, saving 8 cost units (about 67%) compared with the direct route. The company should send packages along S -> A -> C -> G. Time complexity is O(b^(1 + floor(C*/e))) and space complexity is the same, where C* is the optimal cost and e is the minimum edge cost.